

One-Day Team Challenge with a Partner Organization

A workflow for delivering a working product in one hack day with a team of 8–10 participants building with AI-assisted development tools, on a challenge supplied by a partner organization.

Context

Attribute	Value
Team size	8–10 participants
Duration	One hack day
Challenge source	Partner organization
Development mode	AI-assisted; each team builds with AI coding tools
Ideation input	Discover output from the FaithTech 4D cycle (Discover, Discern, Develop, Demonstrate)
Build iteration	FaithTech co-creation cycle (Request, Receive, Review, Render, Rejoice) within each team

Roles and working agreement

The hack day operates as a compressed product delivery cycle with three roles:

Role	Product analogue	Responsibility
Partner organization	Customer	Supplies the challenge and domain context; receives the product and continues to iterate on it after the event
Facilitator	Product owner	Sets product direction, assigns work, and decides what continues beyond the hack day
Participants	Development team	Build the assigned capabilities with AI assistance

The facilitator carries three decisions on behalf of the partner organization:

- Product direction — what the product does, resolved through the PRD (Phase 2)
- Work assignment — who builds what, resolved through capability assignment (Step 10)
- Continuation — which work completes within the hack day and which moves into a follow-on development cycle that iterates with the partner organization after the event

A working agreement — the hacking agreement — shall record this role mapping and shall be understood by all three parties before the build starts. The agreement states:

- The role mapping and the facilitator's decision authority
- The scope the hack day is expected to deliver
- The disposition of work after the event: which artifacts transfer to the partner organization and which move into the follow-on development cycle

Workflow at a glance

Phase	Steps	Output
1. Discovery and ideation	1–2	Consolidated idea set
2. Definition	3–7	Low-fidelity PRD, selected starter template, developed specs, PRD chunked into core capabilities, and cross-capability contracts defined
3. Repository and team setup	8–13	Central repository with committed contract skeleton, CI pipeline, protected main, and per-team feature branches
4. Build and integration	14–15	Working product on main

Phase 1 — Discovery and ideation

Step 1. Brainstorm in pairs

Brainstorm all product details in pairs, grounded in the Discover output of the FaithTech 4D cycle — gathered directly with the partner organization when present, or taken from the notes the partner left.

Keep the medium low fidelity:

- Use a notepad.
- Jot ideas down as they surface.
- Defer structure and polish.

Low-fidelity capture keeps the pace high within the timebox and prevents premature convergence on a single pair's framing.

Step 2. Consolidate as a group

Share each pair's notes with the full group and consolidate them into a single idea set.

Phase 2 — Definition

Step 3. Derive a low-fidelity PRD

Derive a very low-fidelity product requirements document (PRD) from the consolidated idea set. The PRD records what the product does; it defers visual and technical detail.

Step 4. Select a starter template

Select a starter template that matches the product shape the PRD describes — for example, a Clean Architecture solution or a Next.js starter. Selecting the template after the PRD lets the choice follow the product the PRD defines rather than anticipate it. A template removes project setup from the critical path and gives AI coding tools an established structure to extend rather than a blank repository to invent.

Step 5. Develop the specs

Develop the specs from the low-fidelity PRD. Elaborate each part of the PRD into a specification precise enough to chunk into capabilities and to derive the cross-capability contracts from — the shared domain entities and their persisted shape, the navigation map across the product, and the behavior each part shall deliver.

The facilitator owns the specs and their evolution during the day: the specs are the single reference every team builds against, and when a spec changes the facilitator announces the change to the affected teams. Developing the specs before the PRD is chunked gives every capability a stable description to build against rather than one still forming as the build starts.

Step 6. Chunk the PRD into core capabilities

Chunk the PRD into core capabilities that teams can build in parallel:

- Modules — slices of domain functionality
- Pages — distinct screens, such as login
- UI — the design system shared across pages

Each capability shall be independently buildable so that teams do not block one another.

Step 7. Define the cross-capability contracts

Independent buildability rests on the boundaries between capabilities being fixed before the teams build against them. For every boundary where one capability consumes another, define the cross-capability contract — the surface the two capabilities exchange — and record it precisely enough that an agent on either side can build against it without consulting the other team.

The product requires the following cross-capability contracts wherever the corresponding boundary exists:

Contract	Boundary it fixes	Owner	Consumers
Data model contract	The shared domain entities and their persisted shape	Facilitator	Every capability that reads or writes shared data
Module API contract	The public surface a module exposes to its callers — types, signatures, and operations	The owning module capability	Pages and modules that call the module
Transport contract	The routes and the request and response schemas crossing the client–server boundary	The capability that owns the endpoint	Every capability that calls the endpoint
Design system contract	The shared components, their props, and the design tokens	The UI capability	Every page
Navigation contract	The route map and the links between pages	Facilitator	Every page
Identity contract	How the current user, session, and authorization propagate, where the product has authentication	The capability that owns authentication	Every capability behind authentication
Spec contract	The PRD chunk boundaries and their ownership	Facilitator	Every capability
Convention contract	The repository-wide agent-guidance file, lint rules, format rules, and test command	Facilitator	Every capability, enforced by the CI pipeline

A cross-capability contract changes rarely and deliberately: every consuming team rebuilds against it when it changes, so the facilitator arbitrates changes and announces them across teams. Contracts internal to a single capability are not defined here; the team that owns the capability defines them under the [Parallel agent-assisted build](#) workflow.

Phase 3 — Repository and team setup

Step 8. Set up a central repository

Create a central git repository from the starter template and push it to the shared host.

Step 9. Commit the cross-capability contract skeleton

Commit the cross-capability contracts to main as code before the capabilities diverge — shared type and entity definitions, module interface declarations, route and schema stubs, design tokens and component signatures, and the repository-wide agent-guidance file. Every team's agents then read the same committed boundary rather than inventing their own. This mirrors, at the capability level, the slice-level scaffolding each team performs later.

Step 10. Assign capabilities

Assign each core capability to a small team or an individual.

Step 11. Create feature branches

Each team creates its own feature branch for its capability.

Step 12. Add collaborators

Add participants as collaborators so that each participant can push to their team's feature branch.

Step 13. Protect main and set up the CI pipeline

Protect the main branch so that changes land only through reviewed pull requests. At the same time, set up the CI pipeline so that every pull request is validated from the first merge onward. The facilitator performs both tasks as one setup activity.

Phase 4 — Build and integration

Step 14. Review and merge pull requests

Teams build their capabilities with AI assistance and open pull requests against main. Reviewers review each pull request and merge it to main.

Step 15. Iterate until complete

Teams repeat the build–review–merge loop until the product is complete or the day ends. Within each team, iteration follows the FaithTech co-creation cycle — five R's: Request, Receive, Review, Render, Rejoice. Work that remains open at the end of the day moves into the follow-on development cycle recorded in the working agreement, where it continues to iterate with the partner organization.

Design notes

- Chunking the PRD by capability lets 8–10 participants work in parallel; AI assistance makes each small team productive enough to own a full capability within the day.
- Developing the specs before the PRD is chunked gives every team a single, stable description of the product to build against, and keeps ownership of that description with the facilitator rather than dividing it across the parallel teams.
- Cross-capability contracts, defined before the teams build and committed before the capabilities diverge, fix the seams between capabilities so that independent buildability holds in practice and not only in intent; the contracts most teams depend on — the design system and the shared data model — should stabilize first and change least.
- A protected main branch with pull request review and a CI pipeline keeps integration quality constant while many branches land in quick succession.
- Low-fidelity artifacts — notepad brainstorm and a minimal PRD — match the one-day timebox; detail beyond what teams need to start building is deferred.